Module-12d Fast Fourier Transform

We have seen in the previous module that DFT plays an important role in digital signal processing. One of the reasons is the existence of efficient algorithms to implement it. In this module we discuss such algorithms

Recall that

$$\text{DFT: } X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} \qquad 0 \leqslant k \leqslant N-1 \tag{1}$$

$$\text{IDFT: } x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn} \qquad 0 \leqslant n \leqslant N-1. \tag{2}$$

Since DFT and IDFT involves similar expressions and the same type of computations, our discussion of efficient computational algorithms for DFT applies to efficient computation of IDFT too. Let us count the number of computations involved in equation (1).

Operation	**For particular k** **e.g. $X[0]$**	**For all N points** $\{X[0], X[1], \cdots, X[k]\}$
Number of complex multiplications	N	$N \times N = N^2$
Number of complex additions	$N-1$	$N \times (N-1) = N^2 - N$

Direct computation of DFT is inefficient primarily because it does not exploit the symmetry and periodicity properties of W_N. Let us look into these two properties.

$$\text{Symmetry: } W_N^{k+\frac{N}{2}} = -W_N^k \quad \text{and} \quad \text{Periodicity: } W_N^{k+N} = W_N^k.$$

Proof: Note that

$$W_N^{k+\frac{N}{2}} = e^{-j\frac{2\pi}{N}(k+\frac{N}{2})} = e^{-j\frac{2\pi}{N}k} e^{-j\frac{2\pi}{N}\frac{N}{2}} = e^{-j\pi} e^{-j\frac{2\pi}{N}k} = -e^{-j\frac{2\pi}{N}k} = -W_N^k.$$

$$W_N^{k+N} = e^{-j\frac{2\pi}{N}(k+N)} = e^{-j2\pi} e^{-j\frac{2\pi}{N}k} = e^{-j\frac{2\pi}{N}k} = W_N^k.$$

We use these properties to formulate algorithms that are collectively known as Fast Fourier Transform (FFT).

For complex-valued sequence $x[n]$ of N points, DFT is given by

$$X_{\mathtt{R}}[k] = \sum_{n=0}^{N-1} \left(x_{\mathtt{R}}[n] \cos\left(\frac{2\pi}{N} kn\right) + x_{\mathtt{I}}[n] \sin\left(\frac{2\pi}{N} kn\right) \right) \tag{3}$$

$$X_{\mathtt{I}}[k] = \sum_{n=0}^{N-1} \left(-x_{\mathtt{R}}[n] \sin\left(\frac{2\pi}{N} kn\right) + x_{\mathtt{I}}[n] \cos\left(\frac{2\pi}{N} kn\right) \right) \tag{4}$$

Equation (3) and equation (4) results in following computations

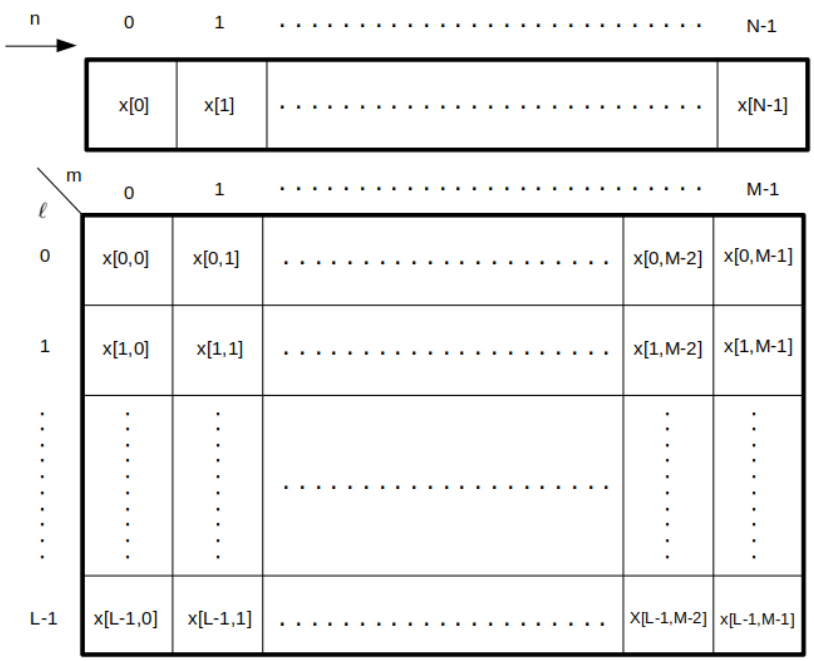
- $2N^2$ evaluation of trigonometric functions.
- $4N^2$ real multiplications.
- $4N(N-1)$ real additions.
- A number of indexing and addressing operations.

The variety of DFT algorithms optimize each of these steps in different ways leading to different computationally efficient algorithms.

1 Divide and conquer approach

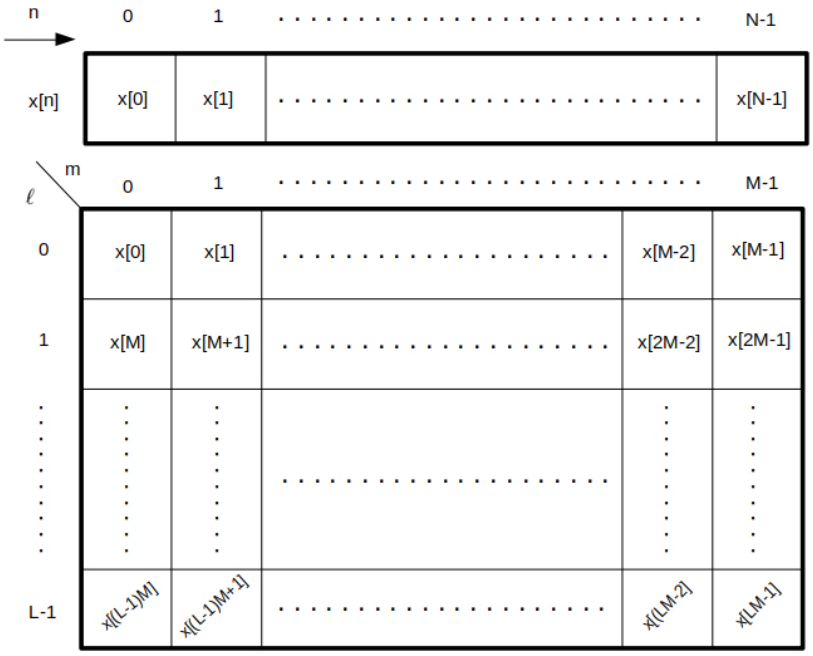
Here the N-point DFT is divided into smaller DFT and then computed. Let N can be factored into two integers, i.e., $N = LM$, where L and M are integers. (5)

This can always be done because if N is a prime number, then we pad the signal with zeros to make N non-prime. Now the sequence $x[n]$ with $0 \leqslant n \leqslant N-1$ can be stored in a two-dimensional array indexed by ℓ and m, where $0 \leqslant \ell \leqslant L-1$ and $0 \leqslant m \leqslant M-1$.

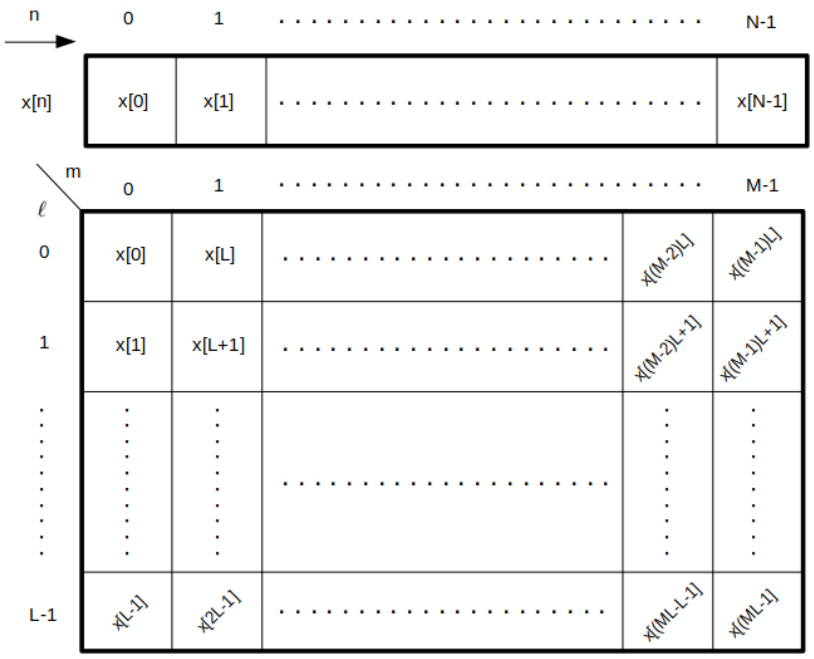


Here ℓ is the row-index and m is the column-index. Now the sequence $x[n]$ can be stored in the rectangular array (given in the figure) in a variety of ways.

1. $n = M\ell + m$: Here the first row consists of first M elements of $x[n]$, the second row the next M elements and so on.

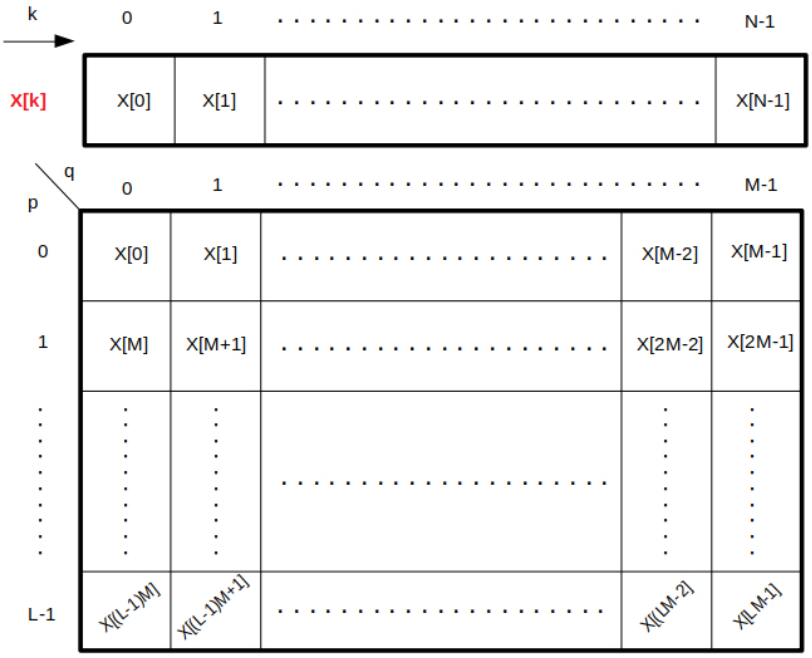


2. $n = \ell + mL$: The first L elements are in first columns, the second column has the next L elements and so on.

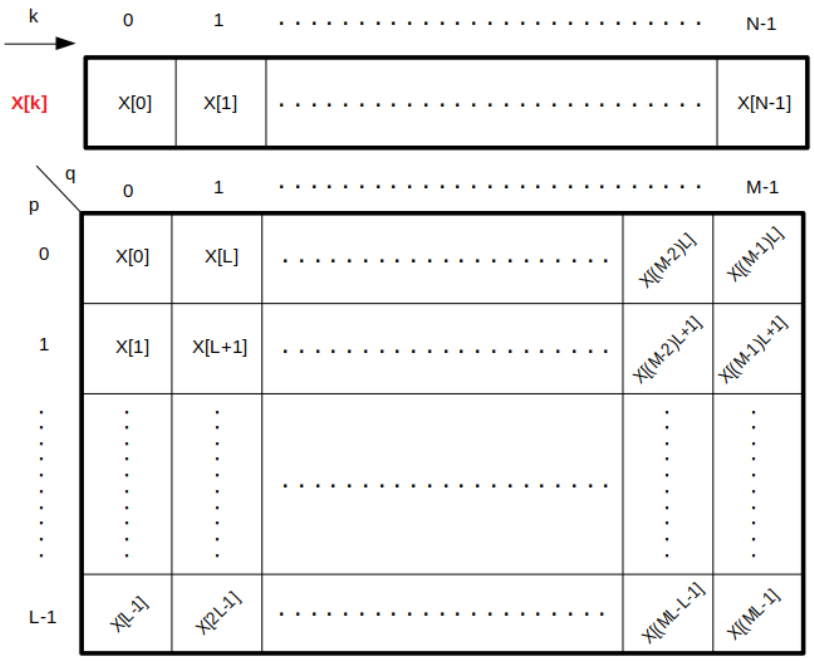


A similar arrangement can be used to store the computed DFT values. In particular, the mapping is from index to pair of indices (p, q), where $0 \leqslant p \leqslant L-1$ and $0 \leqslant q \leqslant M-1$. We can choose one of the following

1. $k = Mp + q$



2. $k = p + qL$



Now suppose $x[n]$ and $X[k]$ are mapped to the arrays $x[\ell, m]$ and $X[p, q]$, respectively. There are four ways of doing this.

Case 1: We use a column-wise mapping $(\ell + mL)$ of $x[n]$ and a row-wise mapping $(Mp + q)$ of $X[k]$. Then DFT equation (1) can be rewritten as –

$$X[p,q] = \sum_{m=0}^{M-1} \sum_{\ell=0}^{L-1} x[\ell, m] W_N^{(Mp+q)(mL+\ell)} \tag{6}$$

Here

$$W_N^{(Mq+p)(mL+\ell)} = W_N^{MLmp} \times W_N^{Mp\ell} \times W_N^{mqL} \times W_N^{q\ell} = W_N^{Nmp} \times W_N^{Mp\ell} \times W_N^{mqL} \times W_N^{q\ell}. \tag{7}$$

By periodicity property, $W_N^{Nmp} = 1$,
$W_N^{mqL} = e^{-j\frac{2\pi}{N}mqL} = e^{-j\frac{2\pi}{N/L}mq} = W_{N/L}^{mq} = W_M^{mq}$ (Since $N/L = M$),
$W_N^{Mp\ell} = e^{-j\frac{2\pi}{N}p\ell M} = e^{-j\frac{2\pi}{N/M}p\ell} = W_{N/M}^{p\ell} = W_L^{mq}$ (Since $N/M = L$).
Using these identities in equation (7), we get

$$W_N^{(Mp+q)(mL+\ell)} = W_L^{p\ell} \times W_M^{mq} \times W_N^{q\ell} \tag{8}$$

Using equation (8) in equation (6), we get

$$X[p,q] = \sum_{m=0}^{M-1} \sum_{\ell=0}^{L-1} x[\ell, m] W_L^{p\ell} W_M^{mq} W_N^{q\ell} = \sum_{\ell=0}^{L-1} \left\{ W_N^{q\ell} \sum_{m=0}^{M-1} x[\ell, m] W_M^{mq} \right\} W_N^{p\ell} \tag{9}$$

Based on equation (9) we have the following steps to compute DFT of $x[n]$:

Step 1: Compute the M-point DFTs

$$F[\ell, q] = \sum_{m=0}^{M-1} x[\ell, m] W_M^{mq} \qquad 0 \leqslant q \leqslant M-1$$

for each of the rows $\ell = 0, 1, \cdots, L-1$. The operation count is given by:

Operation	**Count**
Number of complex multiplications	$LM^2 = NM$
Number of complex additions	$LM(M-1) = NM - N$

On rewriting equation (9), we get

$$X[p,q] = \sum_{\ell=0}^{L-1} \left\{ W_N^{q\ell} F[\ell, q] \right\} W_N^{p\ell} \tag{10}$$

Step 2: We compute a new rectangular array

$$G[\ell, q] := W_N^{\ell q} F[\ell, q] \qquad \begin{cases} 0 \leqslant \ell \leqslant L-1, \\ 0 \leqslant q \leqslant M-1. \end{cases}$$

On rewriting equation (10), we get

$$X[p,q] = \sum_{\ell=0}^{L-1} G[\ell, q] W_N^{p\ell} \tag{11}$$

The total operation count for this step is:

Operation	Count
Number of complex multiplications	$LM = N$
Number of complex additions	0

Step 3: Compute L-point DFT for each $q = 0, 1, \cdots, M-1$ of array $G[\ell, q]$:

$$X[p,q] = \sum_{\ell=0}^{L-1} G[\ell, q] W_L^{\ell p}.$$

The total operation count for this step is:

Operation	Count
Number of complex multiplications	$ML^2 = NL$
Number of complex additions	$LM(L-1) = LN - N$

Total computational complexity of the above method:

Operation	Divide and Conquer	Direct computation
Number of complex multiplications	$N(M+L+1)$	N^2
Number of complex additions	$N(M+L-2)$	$N(N-1)$

Table 1: Comparison of the total number of operations to compute DFT using divide-and-conquer algorithms and direct computation using (1)

From the above table it does not look like we have a lot of improvement in the number of operations. But that is not the case. Let us take an example. Say $N = 1000$ and $L = 2$, $M = 500$ so that $LM = N$. Then on counting the operations using Table 1, we get

Operation	Divide and Conquer	Direct computation
Number of complex multiplications	5.03×10^5	10^6
Number of complex additions	5×10^5	9.99×10^5

When N can be factored into a product of prime numbers of the form $N = r_1 \times r_2 \times \cdots \times r_\beta$, then the decomposition above can be repeated $\beta - 1$ times. This procedure results in smaller DFTs.

Case 2: If we chose to store the input row-wise ($n = M\ell + m$) and the DFT column-wise ($k = qL + p$) then we will have similar to Case 1:

$$X[p,q] = \sum_{m=0}^{M-1} \sum_{\ell=0}^{L-1} x[\ell, m] W_N^{pm} W_L^{p\ell} W_M^{qm} = \sum_{m=0}^{M-1} W_M^{mq} \left[\sum_{\ell=0}^{L-1} x[\ell, m] W_L^{\ell p} \right] W_N^{mp}.$$

The two algorithms have the same complexity. However, they differ in arrangement of computation. Try to figure out the other two cases.

Next we restrict the size of DFT to power of 2.

2 Radix-2 FFT

The previous approach is very efficient when N is highly composite, i.e., N can be factored into many primes.

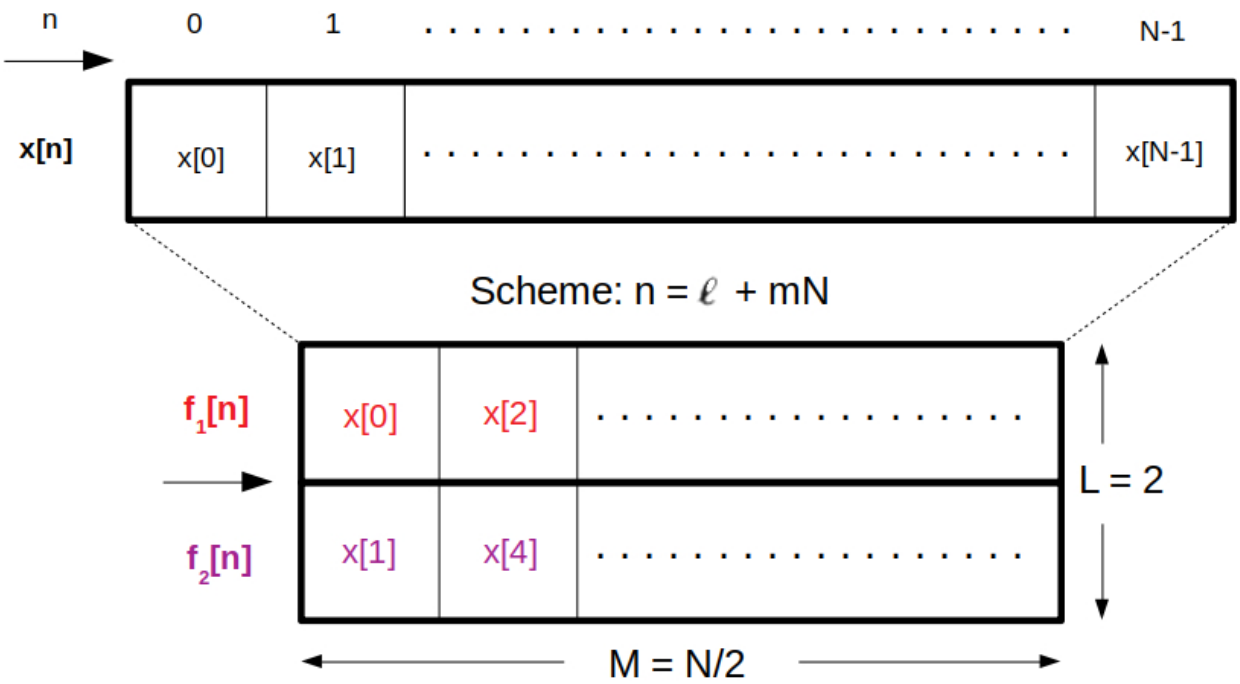
$$N = r_1 \times r_2 \times \cdots \times r_\beta, \quad r_i \text{ are prime numbers.}$$

Of particular importance is the case when $r_1 = r_2 = \cdots = r_\beta =: r$ (say). Then $N = r^\beta$. In such a case the DFTs are of size r and the number β is called the radix of the FFT.

Let us consider $r = 2$, i.e., $N = 2^\beta$. We apply divide and rule method next. We can choose either $L = 2$, $M = \frac{N}{2}$ or $L = \frac{N}{2}$, $L = 2$. We study both the schemes next.

2.1 Decimation-in-time (DIT) Radix-2 FFT

Let $L = 2$ and $M = \frac{N}{2}$. This results in splitting the signal $x[n]$ into two $\frac{N}{2}$ point sequences - say $f_1[n]$ and $f_2[n]$. We are using the scheme $n = \ell + mL$.



Clearly, $f_1 = x[2n]$ and $f_2[n] = x[2n+1]$ for $0, 1, \cdots, \frac{N}{2} - 1$. Thus, $f_1[n]$ and $f_2[n]$ are obtained by decimating $x[n]$ by a factor of 2, and hence the resulting FFT algorithm is called *decimation-in-time* algorithm. Note that if we would have applied the scheme $\ell + mL$, then

$f_1[n]$ and $f_2[n]$ would not have been the even and odd part of $x[n]$. However, the mathematics that follows next will go through for that scheme as well. For our convenience, I am using the scheme $n = M\ell + m$ for explanation. We start the analysis by rewriting equation (1).

$$\begin{aligned}
X[k] = \sum_{n=0}^{N-1} x[n]W_N^{kn} &= \sum_{n \text{ even}} x[n]W_N^{kn} + \sum_{n \text{ odd}} x[n]W_N^{kn} = \sum_{m=0}^{N/2-1} x[2m]W_N^{2km} + \sum_{m=0}^{N/2-1} x[2m+1]W_N^{2m+1k} \\
&= \sum_{m=0}^{N/2-1} x[2m]W_{N/2}^{km} + \left[\sum_{m=0}^{N/2-1} x[2m+1]W_{N/2}^{m}\right] W_N^k \\
&= \sum_{m=0}^{N/2-1} f_[m]W_{N/2}^{mk} + W_N^k \sum_{m=0}^{N/2-1} f_2[m]W_{N/2}^{m} \\
&= F_1[k] + W_N^k F_2[k] \qquad k = 0, 1, \cdots, (N/2) - 1, \qquad (12)
\end{aligned}$$

where $F_1[k]$ and $F_2[k]$ are $\frac{N}{2}$ point DFTs of $f_1[m]$ and $f_2[m]$, respectively. Thus, we have obtained $\frac{N}{2}$ points of DFT. Note that we started with N-point DFT. So, we are yet to evaluate another $\frac{N}{2}$-points.

Note that since $F_1[k]$ and $F_2[k]$ are periodic with period $\frac{N}{2}$, we have

$$F_1\left[k + N/2\right] = F_1[k] \quad \text{and} \quad F_2[k + N/2] = F_2[k].$$

Using equation (12), we have

$$X[k + N/2] = F_1[k + N/2] + W_N^{k+\frac{N}{2}} F_2[k + N/2] \qquad \text{for } k = 0, 1, \cdots, (N/2) - 1. \qquad (13)$$

Using the symmetry property, i.e., $W_N^{k+N/2} = -W_N^k$, in equation (12), we have

$$X[k + N/2] = F_1[k] - W_N^k F_2[k].$$

Thus, the N-point DFT with $k = \{0, 1, 2, \cdots, N-1\}$ is given by

$$X[k] = F_1[k] + W_N^k F_2[k], \qquad (14)$$

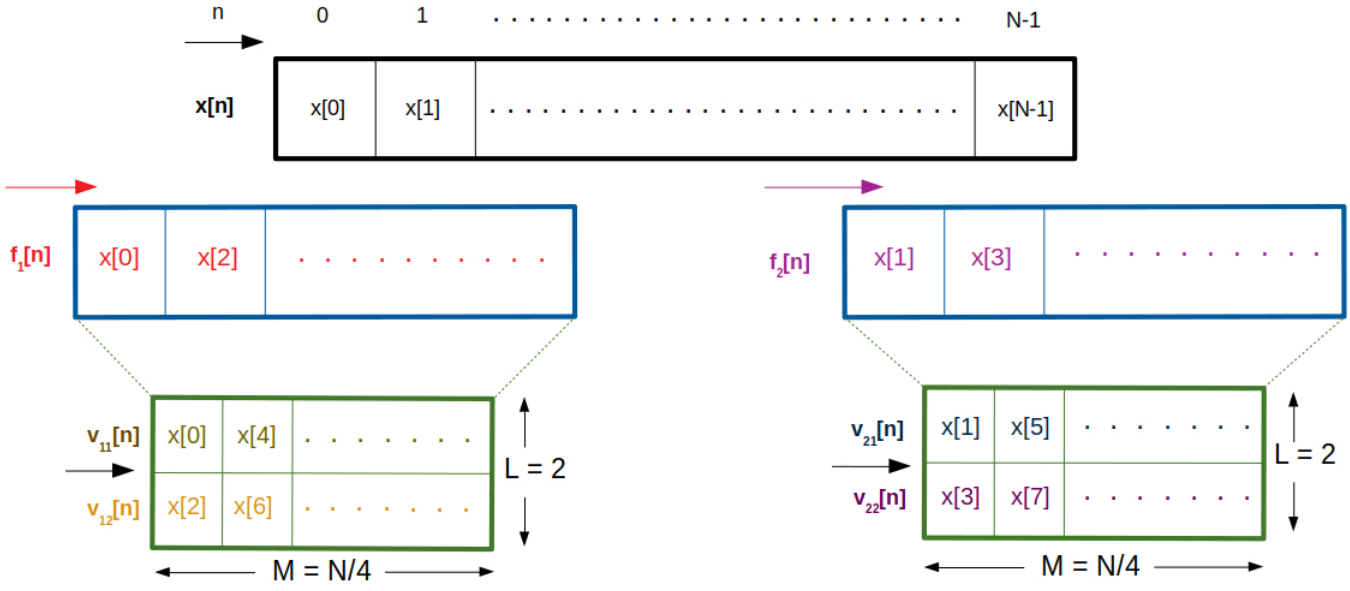
$$X[k + N/2] = F_1[k] - W_N^k F_2[k]. \qquad (15)$$

Having done the decimation-in-time once, we can repeat the process for each sequence $f_1[n]$ and $f_2[n]$. Thus, $f_1[n]$ would result in the following two $\frac{N}{4}$ sequences –

$$v_{11}[n] = f_1[2n] \qquad \text{and} \qquad v_{12}[n] = f_1[2n+1] \quad n = 0, 1, \cdots, N/4 - 1.$$

and $f_2[n]$ gives

$$v_{21}[n] = f_2[2n] \qquad \text{and} \qquad v_{22}[n] = f_2[2n+1] \quad n = 0, 1, \cdots, N/4 - 1.$$



Therefore, using the fact that $W_{N/r}^{k} = W_{N}^{rk}$, we have

$$F_1[k] = V_{11}[k] + W_{N/2}^{k} V_{12}[k] = V_{11}[k] + W_{N}^{2k} V_{12}[k] \tag{16}$$

$$F_2[k] = V_{21}[k] + W_{N/2}^{k} V_{22}[k] = V_{21}[k] + W_{N}^{2k} V_{22}[k] \tag{17}$$

$$F_1[k+N/4] = V_{11}[k] - W_{N/2}^{k} V_{12}[k] = V_{11}[k] - W_{N}^{2k} V_{12}[k] \tag{18}$$

$$F_2[k+N/4] = V_{21}[k] - W_{N/2}^{k} V_{22}[k] = V_{21}[k] - W_{N}^{2k} V_{22}[k], \tag{19}$$

$$\text{for } k = \{0, 1, \cdots, \frac{N}{4} - 1\}.$$

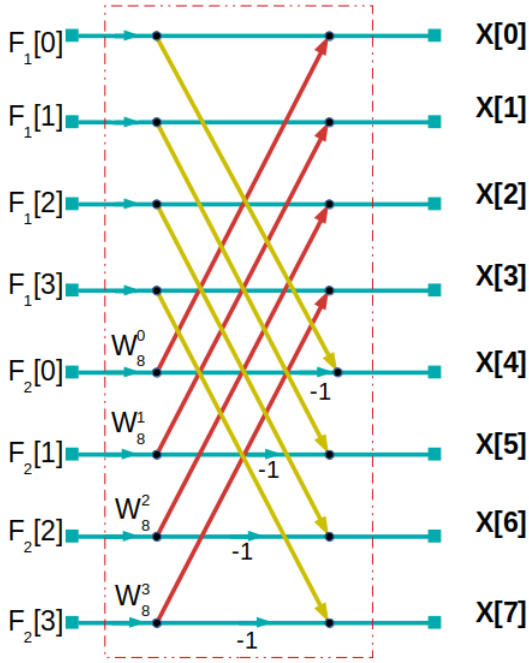
The decimation-in-time can be repeated again and again until the resulting sequences are reduced to one-point sequence.

8-point DFT: Let us consider the case $N = 8$ and pictorially observe how the radix-2 decimation-in-time FFT algorithm is implemented. Consider an 8-point sequence $x[n]$ and let the DFT of $x[n]$ is given by $X[k]$ for $k = 0, 1, \cdots, 7$. Using equation (14), we have

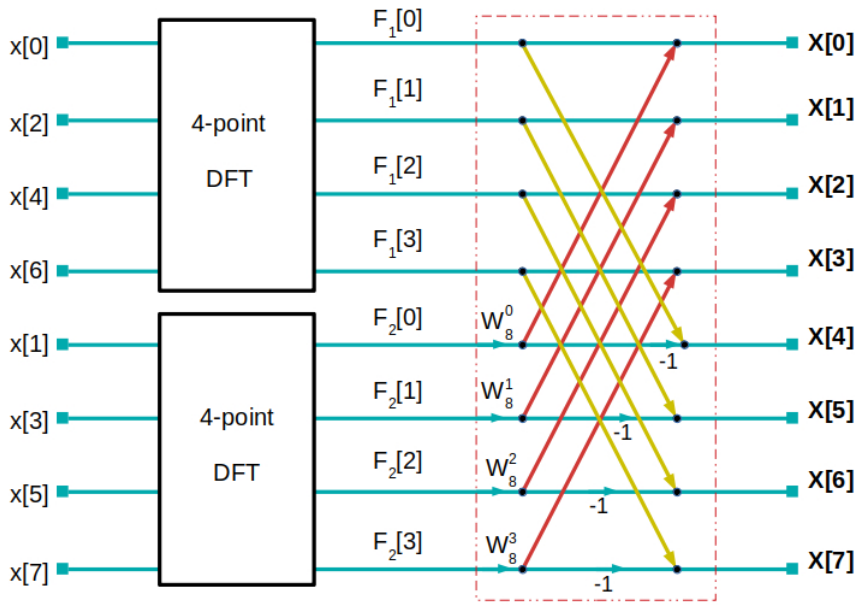
$$X[0] = F_1[0] + W_8^0 F_2[0]$$
$$X[1] = F_1[1] + W_8^1 F_2[1]$$
$$X[2] = F_1[2] + W_8^2 F_2[2]$$
$$X[3] = F_1[3] + W_8^3 F_2[3].$$

Using equation (15), we further have

$$X[4] = F_1[4] - W_8^0 F_2[4]$$
$$X[5] = F_1[5] - W_8^1 F_2[5]$$
$$X[6] = F_1[6] - W_8^2 F_2[6]$$
$$X[7] = F_1[7] - W_8^3 F_2[7].$$



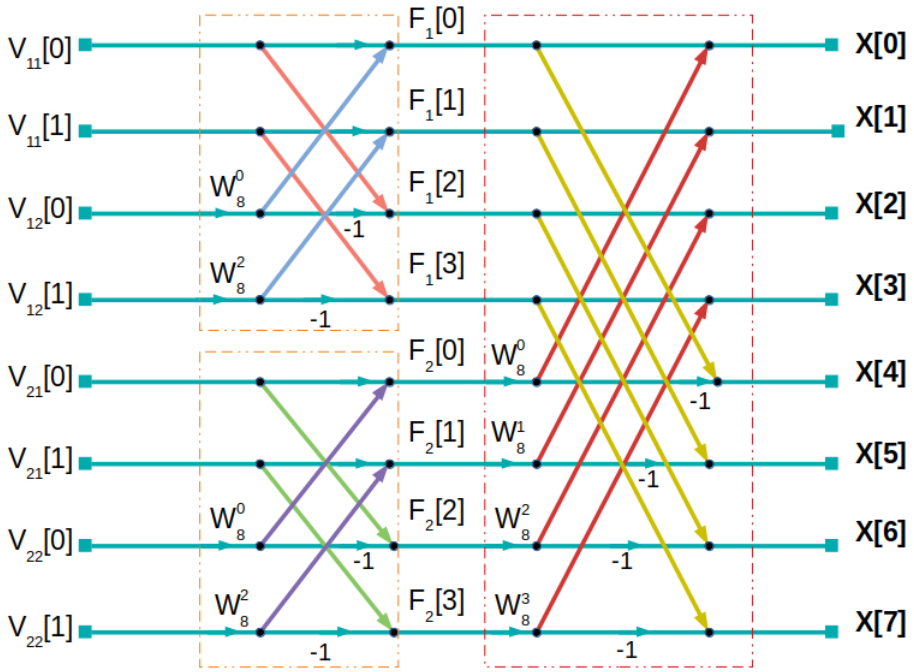
Note that $F_1[k]$ and $F_2[k]$ are DFTs of $f_1[n]$ and $f_2[n]$. Hence, a complete picture would be as follows:



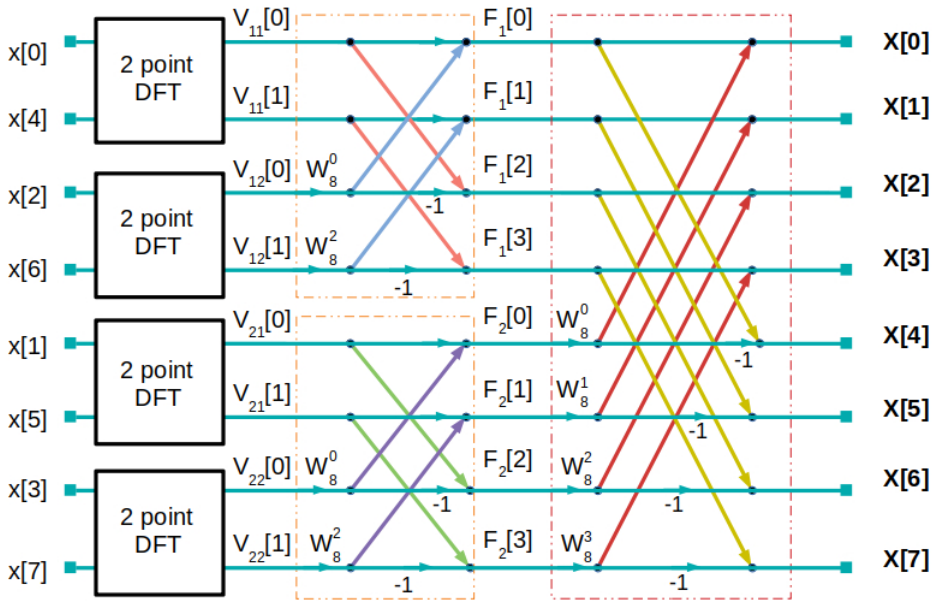
Next we apply divide-and-conquer rule again. We have from equations (16) - (19), the following:

$$F_1[0] = V_{11}[0] + W_8^0 V_{12}[0] \qquad F_2[0] = V_{21}[0] + W_8^0 V_{22}[0]$$
$$F_1[1] = V_{11}[1] + W_8^2 V_{12}[1] \qquad F_2[1] = V_{21}[1] + W_8^2 V_{22}[1]$$
$$F_1[2] = V_{11}[0] - W_8^0 V_{12}[0] \qquad F_2[2] = V_{21}[0] - W_8^0 V_{22}[0]$$
$$F_1[3] = V_{11}[1] - W_8^2 V_{12}[1] \qquad F_2[3] = V_{21}[1] - W_8^2 V_{22}[1].$$

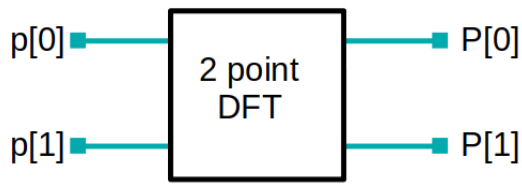
A pictorial representation of the above equations is as follows:



Again, $V_{11}[k]$, $V_{12}[k]$, $V_{21}[k]$, $V_{22}[k]$ are DFTs of $\{x[0], x[4]\}$, $\{x[2], x[6]\}$, $\{x[1], x[5]\}$, and $\{x[3], x[7]\}$, respectively. Hence, a complete picture would be



Let us analyze what are the equations for the 2-point DFTs. Suppose we want to find the 2-point DFT of $\{p[0], p[1]\}$ and let the DFT be $\{P[0], P[1]\}$.



Then using equation (1) we have

$$P[k] = \sum_{n=0}^{1} p[n] W_2^{kn} = \sum_{n=0}^{1} p[n] W_8^{4kn} \qquad \text{Since } W_N^{rkn} = W_{N/r}^{kn} \text{ for } r \neq 0.$$

Thus, we have

$$P[0] = p[0]W_8^0 + p[1]W_8^0 = p[0] + p[1]W_8^0. \tag{20}$$

We are deliberately writing one of W_8^0 in equation (20) as 1 and leaving the other.

$$P[1] = p[0]W_8^0 + p[1]W_8^4 = p[0] - p[1]W_8^0. \tag{21}$$

We have used the symmetry property: $W_8^4 = W_8^{0+8/2} = -W_8^0$ in equation (21). Thus, using equations (20) and (21), we have the following structure for a radix-2 DIT 8-point DFT:

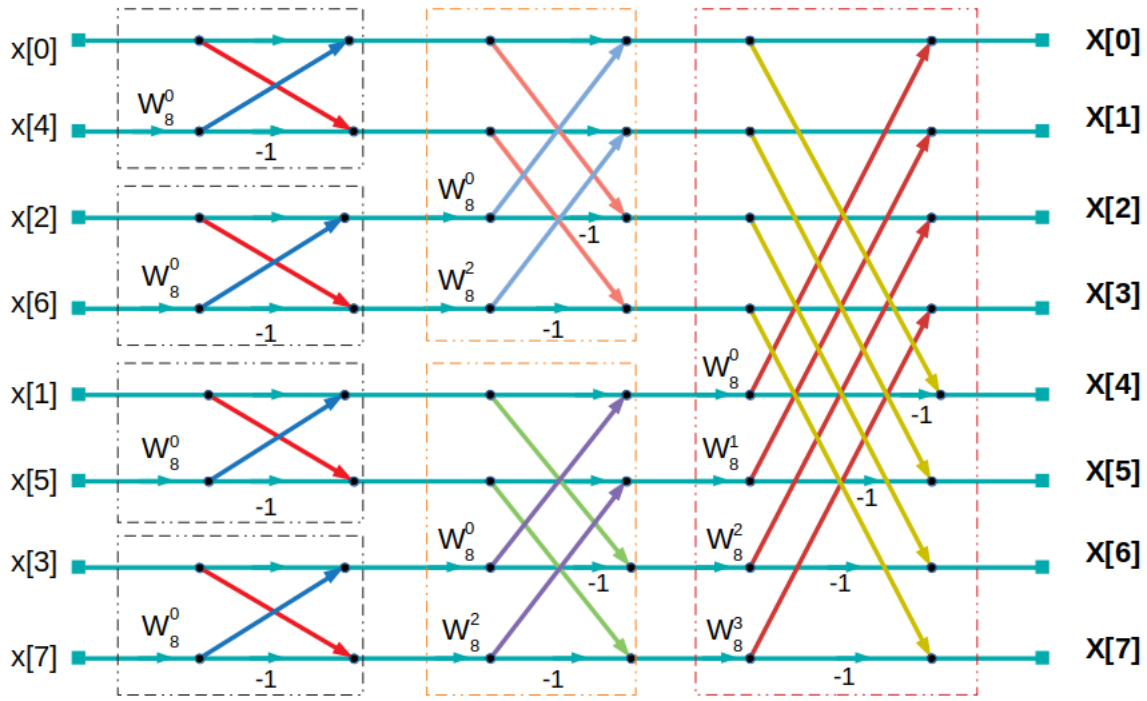
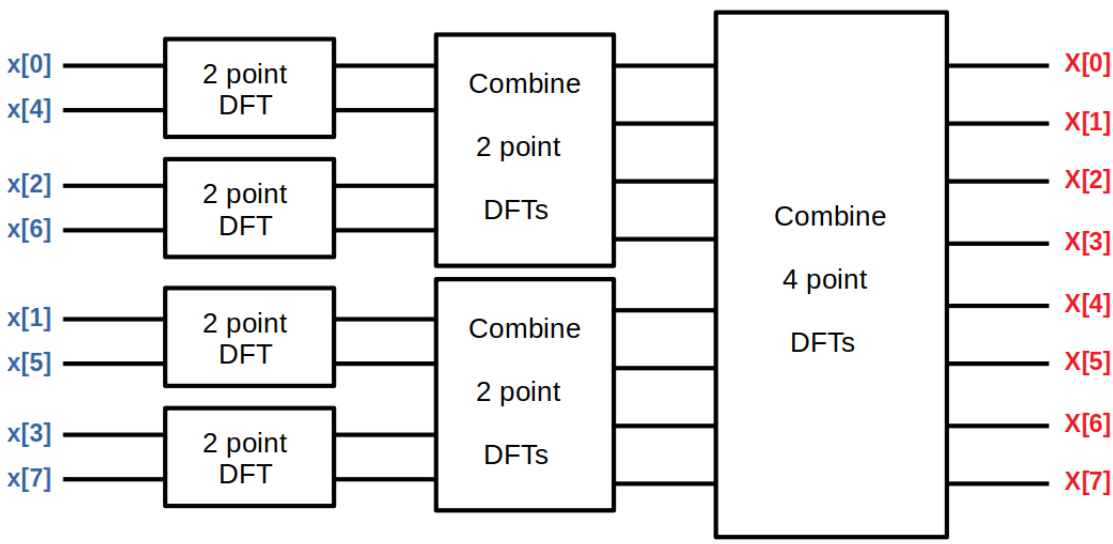
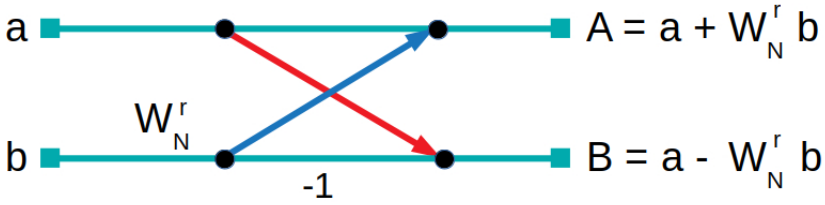


Figure 1: Radix-2 DIT 8-point DFT butterfly structure

Observe that the computation can is done in three stages –



Observe that the basic computation performed at every stage is to take two complex numbers, say (a, b), multiply b by W_N^r and then add and subtract the product from a to form two new complex numbers (A, B).



This basic computation is called a *butterfly* because the flow graph resembles a butterfly. Note that in general, each butterfly involves one complex multiplication $(W_N^r b)$ and two complex additions (the ones that compute A and B). For $N = 2^\beta$, there are $\frac{N}{2}$ butterflies per stage of the computation process and $\log_2 N$ stages. Therefore, the total number of complex multiplications is $\frac{N}{2} \log_2 N$ and complex additions is $N \log_2 N$.

Operation	**Divide and Conquer**	**Direct computation**	**Radix-2 DIT**
# complex multiplications	$N(M + L + 1)$	N^2	$\frac{N}{2} \log_2 N$
# complex additions	$N(M + L - 2)$	$N(N - 1)$	$N \log_2 N$

If we consider the example of $N = 1024$, then we have the following operation count.

Operation	**Direct computation**		**Radix-2 FFT algorithm (DIT)**	
	Count	Time taken in seconds (One operation $= 1\mu s$)	Count	Time taken in seconds (One operation $= 1\mu s$)
Complex multiplications	1048576	1.048576	5120	0.00512
Complex additions	1047552	1.047552	10240	0.01024

Clearly, FFT algorithms are must faster than direct computation (this is why we used $N = 8$ in Example 1.2 of Module 12c). There are other advantages with FFT algorithms. One of them being the storage required for implementing the algorithms. Note that once a butterfly operation is performed on a pair (a, b) to produce (A, B), we no longer need to same (a, b). Hence, we can just replace (a, b) with (A, B). Consequently, at each stage only $2N$ storage registers are required to store the results of the computation.Further, the same $2N$ locations are used throughout the computation of the N-point DFT. Hence, the computations are said to be done *in place*. Why the factor 2 in register computation? Because we are dealing with complex numbers and a single complex number has a real and imaginary part to be stored.

One thing that you should be thinking (if you are not yet) is: how do you arrange the input? Observe that the input is not arranged in ascending order of the index in Figure 1. Well there

is an elegant way to figure that out. Write the index of the signal points in binary form and reverse them to get the order in which you need to arrange the data. We show this for $N = 8$.

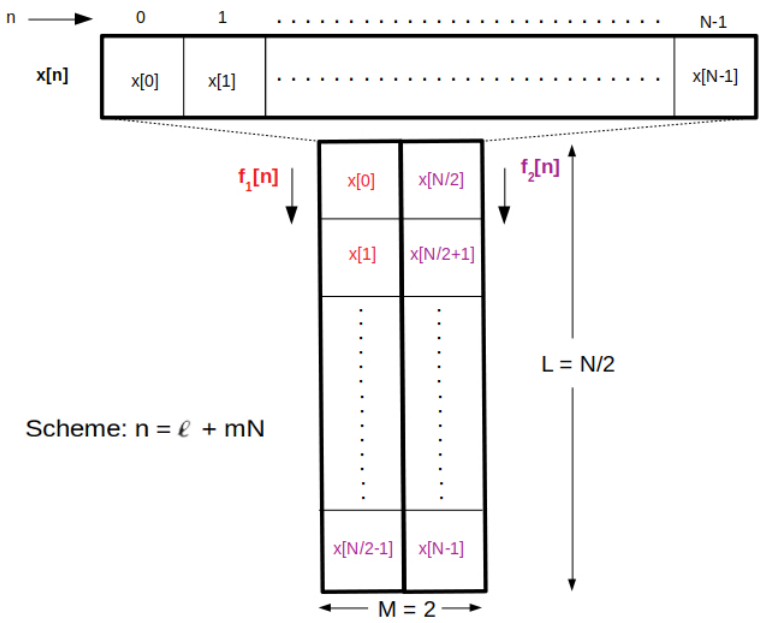
Signal point	**Index n**	**Binary form of index**	**Reverse order**	**Position**
$x[0]$	0	000	000	0
$x[1]$	1	001	100	4
$x[2]$	2	010	010	2
$x[3]$	3	011	110	6
$x[4]$	4	100	001	1
$x[5]$	5	101	101	5
$x[6]$	6	110	011	3
$x[7]$	7	111	111	7

Final arrangement of signal points
$x[0]$
$x[4]$
$x[2]$
$x[6]$
$x[1]$
$x[5]$
$x[3]$
$x[7]$

Thus, the input needs to be stored in bit-reversed order. Then performing the butterfly computations give us the DFT sequence in the natural order. Next we look into the second scheme in radix-2 FFT algorithms.

2.2 Decimation-in-frequency (DIF) Radix-2 FFT

Let $L = \frac{N}{2}$ and $M = 2$. Hence the sequence $x[n]$ will be segmented in the following way:



Using the DFT equation (1), we have

$$X[k] = \sum_{n=0}^{N/2-1} x[n]W_N^{kn} + \sum_{n=N/2}^{N-1} x[n]W_N^{kn} = \sum_{n=0}^{N/2-1} x[n]W_N^{kn} + \sum_{n=0}^{N/2-1} x[n+N/2]W_N^{k(n+N/2)}$$
$$= \sum_{n=0}^{N/2-1} x[n]W_N^{kn} + W_N^{kN/2} \sum_{n=0}^{N/2-1} x[n+N/2]W_N^{kn}$$

Note that $W_N^{kN/2} = (W_N^{N/2})^k = (-1)^k$, since $W_N^{N/2} = -1$. Thus, we have

$$X[k] = \sum_{n=0}^{N/2-1} x[n]W_N^{kn} + (-1)^k \sum_{n=0}^{N/2-1} x[n+N/2]W_N^{kn} = \sum_{n=0}^{N/2-1} \left\{x[n] + (-1)^k x[n+N/2]\right\} W_N^{kn}.$$

Now let us split $X[k]$ into even and odd-numbered samples. For $k = 0, 1, \cdots, N/2-1$

$$X[2k] = \sum_{n=0}^{N/2-1} \left\{x[n] + (-1)^{2k} x[n+N/2]\right\} W_N^{2kn}$$
$$= \sum_{n=0}^{N/2-1} \left\{x[n] + x[n+N/2]\right\} W_{N/2}^{kn} \tag{22}$$
$$X[2k+1] = \sum_{n=0}^{N/2-1} \left\{x[n] + (-1)^{(2k+1)} x[n+N/2]\right\} W_N^{(2k+1)n}$$
$$= \sum_{n=0}^{N/2-1} \left\{(x[n] - x[n+N/2])\, W_N^n\right\} W_{N/2}^{kn}. \tag{23}$$

Define

$$g_1[n] := x[n] + x[n+N/2] \tag{24}$$
$$g_2[n] := (x[n] - x[n+N/2])\, W_N^n. \tag{25}$$

Then, the $\frac{N}{2}$-point DFT of $g_1[n]$ and $g_2[n]$ are given by equation (22) and equation (23), respectively.

$$X[2k] = \sum_{n=0}^{N/2-1} g_1[n]W_{N/2}^{kn} \quad \text{and} \quad X[2k+1] = \sum_{n=0}^{N/2-1} g_2[n]W_{N/2}^{kn}.$$

We can keep on continuing this, similar to the DIT case, till we are left with 2-point DFT. Finally, the 8-point DFT flow diagram for the DIF case will look like that in Figure 2. I leave it to you to find, how we arrive at the butterfly structure in Figure 2. The basic butterfly structure in this case is given in Figure 3. Clearly, from Figure 2 and Figure 3, the number of computation required is the same as that of the DIT case. Here the inputs are in the natural order and the DFT sequence is in bit-reversed form. Further, the computations are done in place.

The algorithms studied in these modules are faster compared to direct computation of DFT. Hence, these algorithms are called Fast Fourier Transform (FFT). There are other FFT

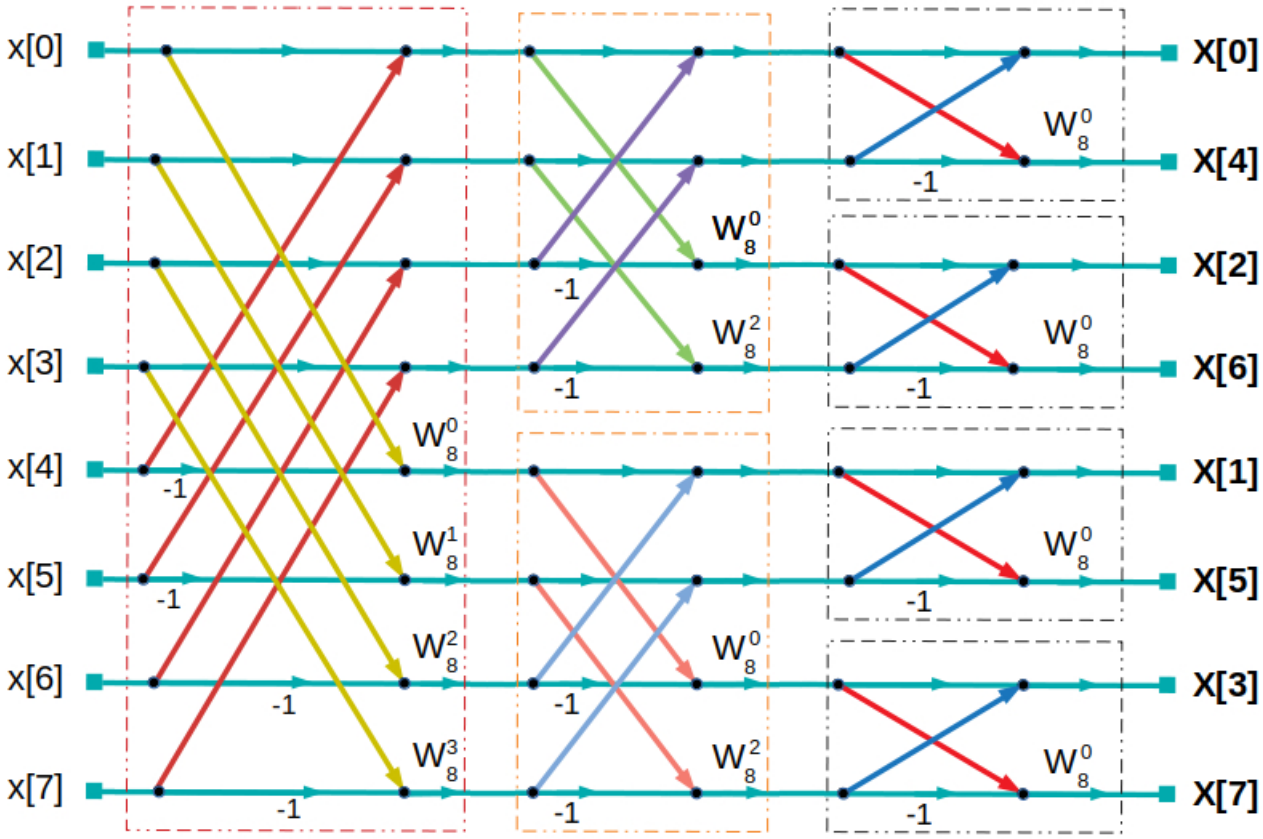


Figure 2: Radix-2 DIF 8-point DFT butterfly structure

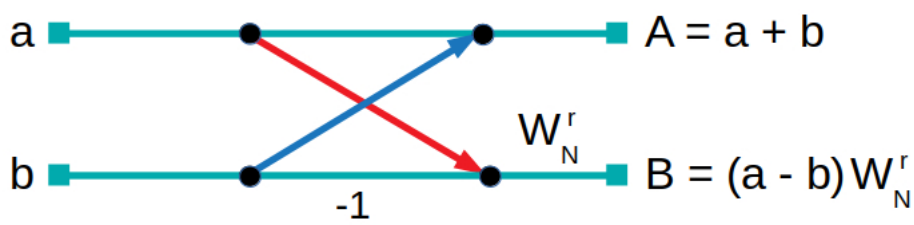


Figure 3: Basic butterfly block in radix-2 DIF 8-point DFT

algorithms as well, e.g., Radix-4 FFT, Split-radix FFT etc. For this course I am limiting myself to the radix-2 algorithm only. For the ones interested in exploring more: you can use the following book: John G. Proakis and Dimitris G. Manolakis, Digital signal processing: principles algorithms and applications, Prentice-Hall Inc.

Apart from DFT computations, the FFT algorithms are used in fast large-integer and polynomial multiplication, efficient matrix-vector multiplication for Toeplitz, circulant and other structured matrices, solving PDEs, computing derivatives etc. I recommend the you-tube videos by Dr. Steve Brunton from University of Washington, Seattle: Channel – Steve Brunton, Link: `https://www.youtube.com/channel/UCm5mt-A4w61lknZ9lCsZtBw/videos`.